LLM Inference Engineering Platform

High-level implementation, measured findings, and charts — Qwen2.5-VL / vLLM on NVIDIA RTX 5090 (32GB)

1. Overview

FastAPI gateway in front of a local vLLM OpenAI-compatible server (Qwen2.5-VL on Windows 11 + WSL2, RTX 5090). The gateway provides admission control, TTFT and latency metrics, GPU telemetry, a live dashboard, optional SLA-based multi-model routing, and benchmark scripts for concurrency, quantization, and prefill/decode timing.

2. High-level architecture

Browser (chat UI + **/dashboard**) → **FastAPI gateway** (async proxy, orchestrator, metrics, router) → **vLLM** (continuous batching) → **RTX 5090**.

Core modules

- **app/orchestrator.py** — bounded concurrency, queue depth, HTTP 429 backpressure, per-request tickets (queued / admitted / first token / completed).
- **app/metrics.py** — NVML GPU telemetry + rolling P50/P95/P99 for wait, TTFT, processing, total latency; tokens/sec.
- **app/router.py** — per-backend orchestrator + metrics; spill to fallback when p95 TTFT breaches SLA.
- **app/main.py** — AsyncOpenAI streaming chat, **/api/metrics**, **/api/metrics/stream**, **/api/router/status**, **/dashboard**.
- **benchmarks/** — load generator, plotting, router/quant/prefill probes; CSVs and PNGs committed under **benchmarks/results/**.

3. Crash under load

At concurrency ≥ 4 with default **--gpu-memory-utilization 0.9**, vLLM crashed and took down the WSL2 GPU VM. With **0.85** and gateway **MAX_CONCURRENCY=8**, the same sweep completed through concurrency 32 with no errors. Overload then showed up as queue wait rather than a process crash.

4. Text concurrency sweep (through the gateway)

Concurrency	Throughput (tok/s)	TTFT p50	TTFT p95	Total p50	GPU util
1	138.3	66.8 ms	104.7 ms	2059 ms	98%
4	547.1	80.7 ms	83.3 ms	2095 ms	98%
8	1043.2	82.3 ms	91.0 ms	2098 ms	98%
16	1043.1	843.4 ms	2203 ms	2867 ms	98%

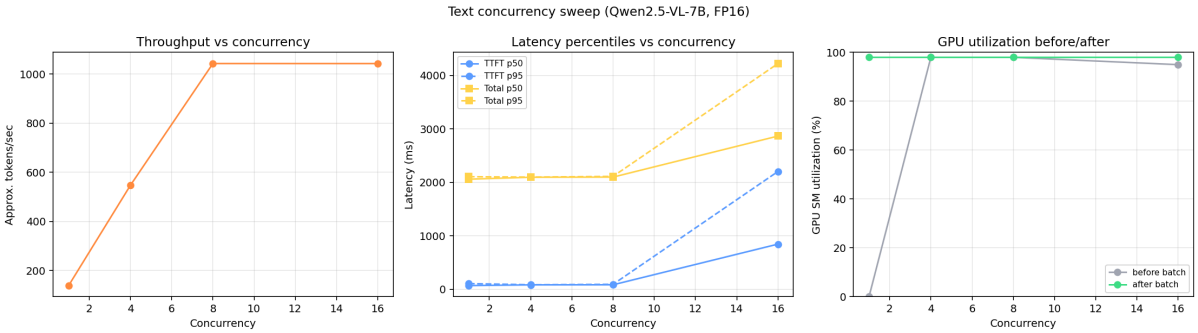

Figure 1 — Text concurrency: throughput vs latency percentiles

Throughput scales roughly linearly from concurrency 1→8, then plateaus at 16 while TTFT rises (queue wait). GPU stays ~98%. The plateau matches the gateway MAX_CONCURRENCY=8 limit.

5. Vision and long-prompt sweeps

Vision (Qwen2.5-VL, one image + prompt)

Concurrency	Throughput (tok/s)	TTFT p50	Total p50
1	107.1	26.7 ms	1541 ms
4	434.2	47.5 ms	1582 ms

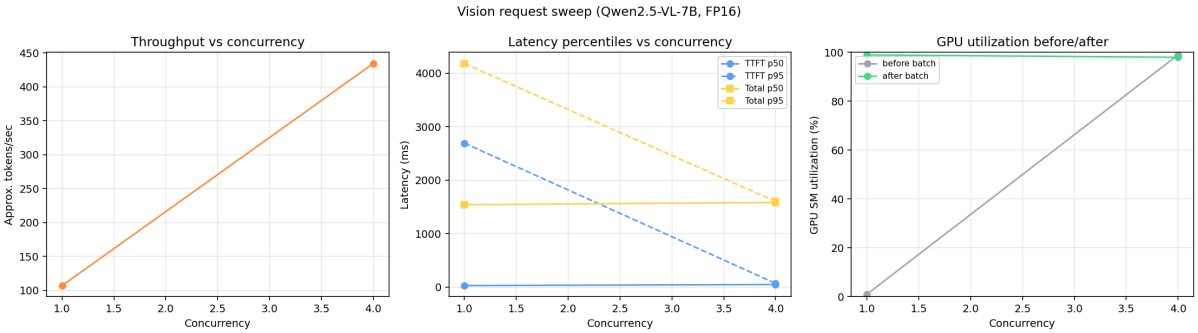

Figure 2 — Vision concurrency sweep

Long prompt (~2,400 input tokens)

Concurrency	Throughput (tok/s)	TTFT p50	Total p50
1	142.1	24.4 ms	1530 ms
4	458.8	37.8 ms	1569 ms
8	695.8	60.7 ms	1627 ms

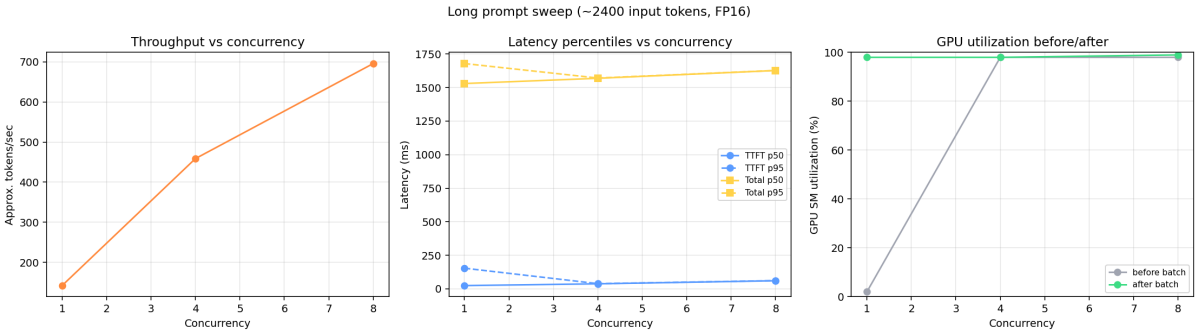

Figure 3 — Long-prompt concurrency sweep

Even with ~8× more input tokens, TTFT stays under ~61 ms at concurrency 8 — prefill is fast enough on this GPU that decode (memory-bandwidth-bound) still dominates total latency.

6. Multi-model SLA routing

Each logical backend has its own orchestrator and metrics. When primary p95 TTFT exceeds its SLA, new requests go to **fast**. Two vLLM engines on this single-GPU WSL2 host crashed the VM, so the load test used two logical backends against one physical engine (same routing code; different URLs on a multi-engine host).

Backend	Requests routed	TTFT p50	TTFT p95	Breaching SLA?
primary (capped)	10	2396 ms	3867 ms	yes
fast (fallback)	30	80.8 ms	464 ms	no

After enough TTFT samples, later requests went to **fast**. Zero errors in 40 requests.

7. Quantization: FP16 vs INT8 (bitsandbytes) vs INT4 (AWQ)

Same model family (**Qwen2.5-7B-Instruct**), same prompts/gateway settings; only weight format changes. Variants loaded one-at-a-time (VRAM / WSL stability).

Variant	c=1 tok/s	c=4 tok/s	c=8 tok/s	TTFT p50 (c=4)	Notes
FP16	131.7	482.6	390.0	83.9 ms	Baseline
INT8 (bnb)	219.9	285.9	272.7	97.6 ms	Memory tool, not speed win
INT4 (AWQ)	93.8*	1080.0	579.4	64.4 ms	Best latency + peak tput

* AWQ c=1 throughput includes a one-time JIT/kernel warm-up (~9 s first request); steady-state TTFT ~60 ms.

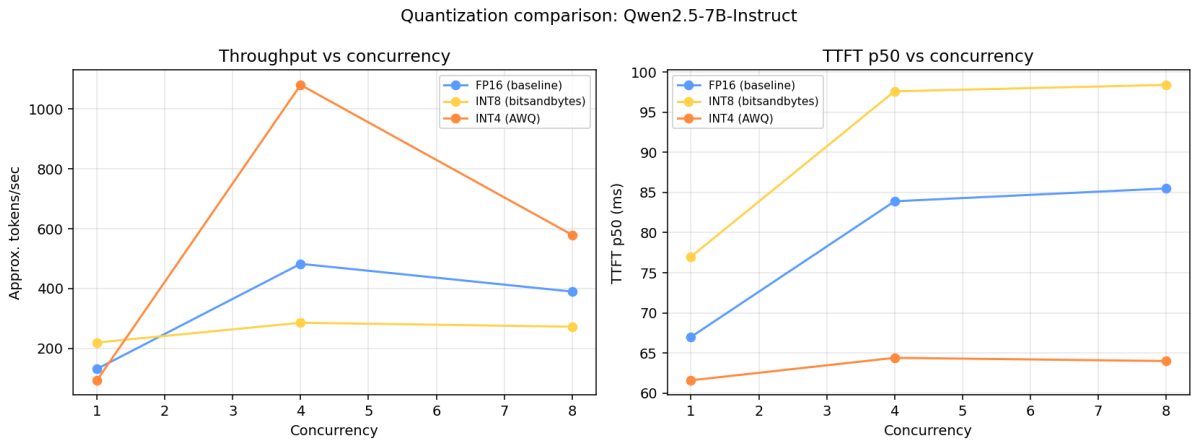

Figure 4 — Quantization comparison

On this GPU, AWQ INT4 had the best latency and peak throughput. bitsandbytes INT8 reduces VRAM vs FP16 but did not improve tokens/sec here.

8. Prefill / decode asymmetry (disaggregation motivation)

Prefill is compute-bound and grows with prompt length; decode is memory-bandwidth-bound with roughly constant per-token cost. Full two-process disaggregation (vLLM **kv_transfer_config**) wants separate GPUs; this host has one, and dual engines already crashed WSL2 here. Below is the measured cost split only.

Prompt tokens	Prefill (ms)	Decode ms / token
300	24.1	15.3

800	30.3	13.9
1800	41.5	15.3
3000	44.9	15.0

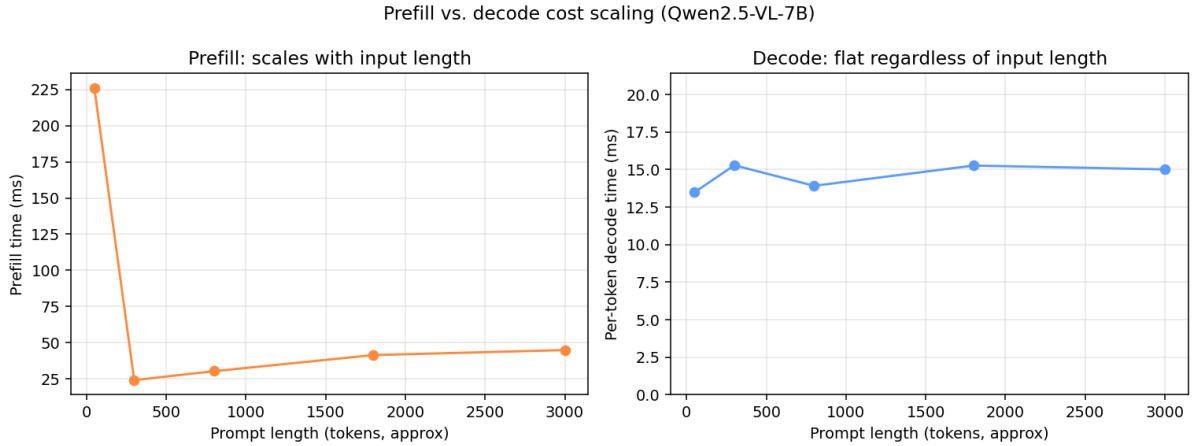

Figure 5 — Prefill time vs per-token decode cost by prompt length

9. Observability surface

- **GET /api/metrics** — GPU util/VRAM/power, queue depth, active requests, tok/s, percentiles.
- **GET /api/metrics/stream** — same payload over SSE (~1 Hz) for the dashboard.
- **GET /dashboard** — live Chart.js UI.
- Streaming chat **done** events include **ttft_ms**, backend name, and degraded flag.

10. Summary

- GPU memory headroom and gateway concurrency limits avoided the crash path under load.
- Throughput scales with concurrency until the admission ceiling; beyond that, queue wait grows while tok/s stays flat.
- Separate wait vs processing vs TTFT in metrics; SLA routing can spill to a fallback backend.
- AWQ INT4 beat FP16 and bitsandbytes INT8 on latency and peak throughput in these runs.
- Prefill cost grows with prompt length; decode ms/token stays roughly flat — motivating split serving when multiple GPUs are available.

Data: benchmarks/results/, RESULTS.md, router/RESULTS.md, quantization/RESULTS.md, disaggregation/RESULTS.md.